Assignment 1: Designing complex data

Goals: Practice designing the representation of complex data.

1.1 Instructions

Be very, very careful with naming! The solution files expect your submissions to be named a certain way, so that they can define their own `Examples` class with which to test your code. Therefore, whenever the assignment specifies:

- the names of classes,
- the names and types of the fields within classes,
- the names, types and order of the arguments to the constructor, or
- filenames,

...be sure that your submission uses exactly those names. Additionally, make sure you follow the course style guidelines. For now the most important ones are: using spaces instead of tabs, indenting by 2 characters, following the naming conventions (data type names start with a capital letter, interfaces begin with an uppercase I, names of fields and methods start with a lower case letter), and having spaces before curly braces.

You will submit this assignment by the deadlines using the course handin server. Follow A Complete Guide to the Handin Server for information on how to use the handin server. **You may submit as many times as you wish. Be aware of the fact that close to the deadline the server may slow down to handle many submissions, so try to finish early.** There will be a separate submission for each problem - it makes it easier to grade each problem, and to provide you with the feedback for each problem you work on.

Practice Problems

Work out these problems from *How to Design Classes* on your own. Save them in an electronic portfolio, so you can show them to your instructor, review them before the exam, use them as a reference when working on the homework assignments.

- Problem 2.4 on page 17
- Problem 3.1 on page 25
- Problem 4.4 on page 34
- Problem 5.3 on page 43
- Problem 10.6 on page 102
- Problem 11.2 on page 113
- Problem 14.7 on page 140

> Everywhere in this assignment that you see *`italic, fixed-width text`*, it is intended to be the name of a field, identifier, class name or interface name you must define...but you likely must modify that name a bit to conform to our Java naming conventions: *`hyphenated-names`* are written in `camelCase`, and *`interface names`* begin with an uppercase `I`.
>
> Everywhere that you see `fixed-width text`, it is exactly the name you must use.

Problem 1: Our Canine Friends

We are designing the data collection for the American Kennel Club. For each dog we need to collect the following information:

- `name`: to be represented as a `String`
- `breed`: of dog to be represented as a `String`
- `yob`: the year of birth given as a four digit number to be represented as an `int`
- `state`: of residence – given as the standard two letter abbreviation to be represented as a `String`
- `hypoallergenic`: a boolean value, `true`, if the dog is hypoallergenic, or `false` if not

Design the class `Dog` that represents the information about each dog for the census.

Make at least three examples of instances of this class, in the class `ExamplesDog`. Two of the examples should be objects named `huffle` and `pearl` and should represent the following two dogs:

- Hufflepuff, a Wheaten Terrier, born in 2012, resides in TX, and is hypoallergenic
- Pearl, a Labrador Retriever, born in 2016, resides in MA, and is not hypoallergenic

---

What to submit

You should submit your data definitions and examples in a file named `Dog.java`

Problem 2: Icecream

Here is a data definition in DrRacket:

```
;; An IceCream is one of:
;; -- EmptyServing
;; -- Scooped

;;An EmptyServing is a (make-empty-serving Boolean)
(define-struct empty-serving (cone))

;;A Scooped is a (make-scooped IceCream String)
(define-struct scooped (more flavor))
```

- Draw the class diagram that represents this data definition. You may draw this as ASCII-art and include it in your submission, if you wish. Or you can just draw it on paper and not submit it. Regardless, we think it will help you in visualizing how the data is organized.
- Convert this data definition into Java. Make sure you use the same names for data types and for the fields, as are used in the DrRacket data definitions, *converted into Java style standards*. Make sure that the constructor arguments are given in the same order as shown.
- Create examples in a class `ExamplesIceCream`. Include in your examples the following two ice cream orders:

  – a cup of ice cream with scoops of `"mint chip"`, `"coffee"`, `"black raspberry"`, and `"caramel swirl"`

  – a cone with scoops of `"chocolate"`, `"vanilla"`, and `"strawberry"`

Hint: This is similar to the Lecture 2 - 2.4 Self-referential unions: Ancestor trees.

Essentially this will creating a list (i.e. your order will be a self-referential ice cream :)).

Make sure the two sample orders given above are named `order1` and `order2`.

Note: the descriptions above are listed in the order that you would order this in real life. Think carefully how this should be represented as data.

What to submit

You should submit your data definitions and examples in a file named `IceCream.java`

Problem 3: Traveling

We've been asked to help build a new medieval adventure game, *Summer Is Coming*. We're trying to figure out the gameplay mechanics, so we're starting with representations for travel around the game world. Players can live in three types of *housing*: a `Hut`, an `Inn`, and a `Castle`.

A `Hut` has a *`capacity`* and the current count of its *`population`*. The population must be less than the capacity.

An `Inn` has a *`name`*, *`capacity`* and the current count of its *`population`* as well as the number of *`stalls`* in its stable. The population must be less than the capacity.

A `Castle` has a *`name`*, the *`family-name`* of the owners, the current count of its *`population`* as well as the number of carriages it can hold in its *`carriage-house`*.

There are 2 types of *`transportation`* in this game:

- `Horse`
- `Carriage`

Each type of transportation should have a *`from`* and a *`to`* housing. Horses also have a *`name`* and a *`color`* (which you may represent using the color's name). They can only go to an inn if there is room in the stables, but they can go to any hut or castle.

Carriages can only carry a limited supply of *`tonnage`* and only travel *`from`* Inns *`to`* Castles or vice versa. When they go to a Castle there must be room for them in the carriage house.

- Define six examples of housing, including:

- `hovel`: Capacity 5, population 1
- `winterfell`: Named `"Winterfell"`, family name `"Stark"`, population 500, can hold 6 carriages
- `crossroads`: Named `"Inn At The Crossroads"`, capacity 40, population 20, 12 stalls

  The others can be whatever you wish.

- Define four types of travel, two of each kind.

Name your examples `horse1`, `carriage2`, etc., and your examples class `ExamplesTravel`.

Please note: We're placing a lot of restrictions on the data, such as the population being less than capacity, possible destinations of carriages, etc. However we aren't (yet) actually enforcing these in the code. The ways to enforce these constraints will be further explored later in the semester. For now, you are expected to create examples that conform to these constraints

---

What to submit

You should submit your data definitions and examples in a file named `Travel.java`